

Contexte et objectif du TP

Dans ce premier TP, nous nous intéressons à la modélisation et la gestion d'une écurie de Formule 1. Plus particulièrement, nous nous intéresserons aux voitures et au personnel de l'écurie. Au cours de ce TP, nous allons donc créer des objets pour gérer une écurie de Formule 1.

Ce TP permet ainsi d'illustrer les notions suivantes :

- les bases du langage Java ;
- les notions de classe, d'attribut, de constructeur et de méthodes ;
- les notions d'encapsulation et de visibilité ;
- les classes composites et composants ;
- les associations 1-1.

Bonnes pratiques à adopter

- Testez votre code au fur et à mesure de votre avancement.
- Respectez les conventions de nommage Java.
- Faites des indentations de manière correcte.
- Donnez des noms compréhensibles et pertinents à vos variables, attributs, classes, méthodes.
- Respectez autant que possible le principe d'encapsulation.
- Utilisez des structures de données adaptées.
- Faites des méthodes courtes avec peu de niveaux d'indentation.
- Ne faites pas de duplication de code, mais privilégiez le code réutilisable.

- Utilisez les outils de votre IDE pour faire du *refactoring* (clic-droit puis 'Refactor' sur IntelliJ IDEA).
- Utilisez la Javadoc pour comprendre le fonctionnement des méthodes que vous appelez.
- Quand c'est nécessaire, commentez votre propre code de façon pertinente au format Javadoc (commentaire commençant par `/**` au-dessus des définitions des attributs et méthodes).

1 Création des voitures et personnel d'une écurie de Formule 1

1.1 Rappels sur les objets et les classes

Pour rappel, une *classe* est la modélisation d'un type de données définissant à la fois son état (comment représenter la donnée) et son comportement (les méthodes permettant la manipulation de la donnée). Ceci signifie qu'une classe est un type d'*objet*, c'est un "moule" à objets. On peut ainsi produire ou créer un exemplaire à partir du moule. On appelle cet exemplaire une *instance* de la classe. Un objet est donc une instance d'une classe.

L'état d'un objet est caractérisé par la valeur de ses *attributs*. Son comportement est défini à travers les *méthodes* spécifiées par sa classe.

Un attribut est une variable servant à mémoriser une caractéristique d'un objet. Ils sont précisés dans la classe et chaque instance de la classe aura son propre exemplaire, avec une valeur potentiellement différente d'un objet à l'autre.

1.2 Diagramme de classe

UML (*Unified Modeling Language*) est un langage visuel pour la spécification et la conception d'un système d'informations. C'est un langage couramment utilisé en conception orientée objet. En particulier, un des types de diagramme d'UML est le *diagramme de classe*, qui permet de représenter les classes intervenant dans un système, ainsi que les liens entre ces classes.

1.3 Création de la classe Moteur

Nous nous intéressons dans un premier temps à la représentation d'un moteur. Le responsable technique de l'équipe de Formule 1 vous explique les choses suivantes :

- un moteur est défini par son carburant (qui peut être essence ou diesel) ainsi que par sa puissance (en *ch*) ;
- les moteurs avec lesquels il travaille ont toujours une puissance supérieure à 600 *ch* ;

- par défaut, on considère que les moteurs sont à essence avec la puissance minimale ;
- il est possible de modifier la puissance d'un moteur mais absolument impossible de changer son carburant.

Question 1. Représentez sous forme de diagramme de classe un moteur avec ses attributs.

Question 2. Après avoir ouvert IntelliJ IDEA, créez un nouveau projet **Formule1**, un paquetage **modele**, et écrivez une classe **Moteur** qui représente le moteur d'une voiture, dont vous définirez les attributs. Commentez chacun des attributs sous format Javadoc.

Question 3. Ajoutez trois constructeurs dans la classe **Moteur** :

- un constructeur par défaut ;
- un constructeur par données ;
- un constructeur par copie.

Assurez-vous que les instances créées par les constructeurs respectent les spécifications d'un moteur. Commentez chaque constructeur au format Javadoc.



IntelliJ IDEA fournit des outils pour vous aider à générer les constructeurs par données (clic-droit puis *Generate...*).

Question 4. Complétez la classe **Moteur** avec les éléments suivants :

- les accesseurs et mutateurs nécessaires ;
- une méthode **toString()** qui renvoie une chaîne de caractères avec les informations sur le moteur.



Notez que la méthode **toString()** est en fait une redéfinition de la méthode **toString()** qui est définie dans la classe **Object**. Ce point sera détaillé lors de la prochaine séance.
IntelliJ IDEA fournit des outils pour vous aider à générer les accesseurs, les mutateurs et la méthode **toString()** (clic-droit puis *Generate...*).

Question 5. Écrivez une méthode principale (**public static void main(String[] args)**) afin de tester l'utilisation de la classe **Moteur**. Assurez-vous de tester tous les cas possibles et de veiller à ce que les préconisations sur la puissance et le carburant soient bien toujours respectées.



IntelliJ IDEA fournit des raccourcis pour vous aider. Tapez *"psvm"* puis *Entrée* pour générer l'entête de la méthode principale. Tapez *"sout"* puis *Entrée* pour faire un affichage sur la console.

1.4 Création de la classe Voiture

Maintenant que nous sommes capables de créer des moteurs, nous allons nous attaquer aux voitures. Le responsable du parc de véhicules de l'écurie de Formule 1 vous donne les explications suivantes :

- chaque voiture est définie par un numéro d'immatriculation, une marque, et bien sûr un unique moteur ;
- le numéro d'immatriculation doit être unique (deux voitures différentes ne peuvent pas avoir le même numéro) et il doit être généré automatiquement lors de la création d'une nouvelle voiture ;
- il n'est pas possible de modifier le numéro d'immatriculation ou la marque d'une voiture ;
- par contre, il est possible de remplacer son moteur.

Question 6. Modifiez le diagramme de classe afin d'y rajouter la classe **Voiture** et le lien avec la classe **Moteur**.

Question 7. Dans le paquetage **modele**, ajoutez une classe **Voiture** avec ses attributs. Accordez une attention particulière à la manière dont vous générez des numéros d'immatriculation uniques, et à la manière dont vous modélisez le fait qu'une voiture est composée d'un moteur.

Question 8. Ajoutez les quatre constructeurs suivants :

- un constructeur par défaut ;
- un constructeur par donnée de la marque ;
- un constructeur par données de la marque, de la puissance et du carburant du moteur ;
- un constructeur par données de la marque et d'un moteur.

Assurez vous que les instances créées par ces constructeurs respectent bien les spécifications d'une voiture.

Question 9. Ajoutez les accesseurs et mutateurs nécessaires, ainsi qu'une méthode **toString()**. Écrivez une méthode principale afin de tester l'utilisation de la classe **Voiture**. Testez entre autre le code suivant :

```
1 Moteur m1 = new Moteur(null);
2 Moteur m2 = new Moteur(1000, 'E');
3 Voiture v1 = new Voiture(null, m1);
4 Voiture v2 = new Voiture("Ferrari", m2);
5 Voiture v3 = new Voiture("Ferrari", m2);
```

```

6 System.out.println("v1 : " + v1.toString());
7 System.out.println("v2 : " + v2.toString());
8 System.out.println("v3 : " + v3.toString());
9 m2.setPuissance(1200);
10 System.out.println("v2 : " + v2.toString());
11 System.out.println("v3 : " + v3.toString());

```

1.5 Création de la classe **Personne**

Pour constituer une bonne écurie de Formule 1, il faut de bons véhicules, mais aussi une bonne équipe de personnes pour conduire et réparer les voitures. Nous nous intéressons donc à présent à la création des objets de type **Personne**. Le responsable de l'équipe vous donne les informations suivantes :

- chaque personne est définie par un numéro d'identification, un nom, un prénom et une adresse ;
- le numéro d'identification doit être unique et doit être généré automatiquement lors de la création d'une nouvelle instance de la classe **Personne** ;
- une personne ne peut pas changer son nom ou son prénom, mais elle peut changer son adresse.

Question 10. Modifiez le diagramme de classe afin d'y rajouter la classe **Personne**.

Question 11. Ajoutez une classe **Personne** avec les éléments suivants :

- ses attributs ;
- un constructeur par défaut ;
- un constructeur par données du prénom, du nom et de l'adresse ;
- les accesseurs et mutateurs nécessaires ;
- une méthode **toString()** ;
- une méthode principale afin de tester l'utilisation de la classe **Personne**.

2 Association 1-1 unidirectionnelle : **Personne** - **Voiture**

Dans les équipes de Formule 1, il existe quelques règles pour associer les voitures et les personnes :

- une personne peut éventuellement conduire une voiture (dans la vie réelle c'est difficile de conduire deux voitures à la fois...) ;

- et évidemment une voiture possède éventuellement un conducteur (là aussi, avez-vous jamais vu une voiture conduite par deux personnes en même temps ?).

Nous supposons ici que chaque personne peut accéder à la voiture qu'elle conduit, mais que les voitures ne peuvent pas accéder à leur conducteur. Ainsi, cela se traduira par une relation unidirectionnelle, navigable dans un seul sens.



On pourrait souhaiter réaliser une association bidirectionnelle, navigable dans les deux sens. Cependant, il faut bien noter que les associations bidirectionnelles sont plus complexes à coder car pour assurer la cohérence de la relation il faut mettre à jour les attributs dans les deux classes correspondantes.

Question 12. Modifiez le diagramme de classe afin d'y rajouter la relation unidirectionnelle entre les voitures et les personnes.

Question 13. Modifiez les classes **Voiture** et **Personne** en y ajoutant les attributs nécessaires afin de modéliser la relation entre ces deux classes.

Si besoin, modifiez également :

- les constructeurs de ces classes (on supposera qu'à la création d'une personne elle ne conduit pas de voiture) ;
- les accesseurs et mutateurs ;
- la méthode **toString**.

Testez vos modifications.

Question 14. Ajoutez dans la classe **Personne** une méthode **public boolean estPieton()** qui indique si la personne est à pied ou non.

Pour le moment, on n'a pas écrit de mutateur pour la voiture conduite par une personne car il faut gérer de manière plus fine la relation entre les deux classes. Nous définissons donc des règles métiers *affectation* et *restitution* d'une voiture pour bien gérer l'association Personne-Voiture. Pour pouvoir affecter une voiture à une personne, nous appliquerons les règles suivantes :

- la voiture doit exister ;
- la personne doit être un piéton (pas encore de voiture affectée).

Si ces deux conditions sont vérifiées, alors on affecte la voiture à la personne.

Question 15. Dans la classe **Personne** définissez une méthode **public boolean affecter(Voiture v)** qui gère l'affectation de la voiture **v** à la personne. Cette méthode renvoie un booléen qui indique si l'affectation a pu être effectuée correctement ou non. Faites des tests pour vérifier que tout se déroule correctement. En particulier, vérifiez ce qui se passe dans les cas suivants :

- sur une personne, on affecte une voiture qui vaut **null** ;
- sur une personne qui conduit déjà une voiture, on affecte une voiture ;
- on affecte une même voiture à deux personnes différentes.

Maintenant que nous savons affecter des voitures à des personnes, il faut qu'une personne puisse restituer la voiture qu'elle conduit.

Ainsi, pour pouvoir restituer la voiture d'une personne, on s'assure juste que la personne n'est pas piéton. Si tel est le cas, on restitue la voiture, et la personne devient donc piéton.

Question 16. Dans la classe **Personne** définissez une méthode **public boolean restituer()** qui restitue la voiture conduite par la personne. Cette méthode renvoie un booléen qui indique si la restitution a pu être effectuée correctement ou non. Faites des tests pour vérifier que tout se déroule correctement. Notamment, vérifiez qu'après avoir restitué une voiture, il est possible d'affecter une nouvelle voiture à la personne.

3 Association bidirectionnelle

Vous avez dû remarquer que l'association unidirectionnelle entre une personne et une voiture pouvait poser quelques problèmes :

- il est possible qu'une même voiture soit affectée à plusieurs personnes ;
- une voiture ne peut pas avoir accès à son conducteur.

Pour remédier à cela, il faudrait mettre en place une association bidirectionnelle.

Question 17. Modifiez la classe **Voiture** en y ajoutant les attributs nécessaires afin de modéliser la relation bidirectionnelle entre les deux classes. Modifiez en conséquence les constructeurs de la classe **Voiture**. Modifiez également la méthode **toString** de cette classe afin de prendre en compte les nouveaux attributs. Testez vos modifications.

Question 18. Ajoutez dans **Voiture** une méthode **public boolean estDisponible()** qui indique si la voiture est disponible (elle n'a pas de conducteur) ou non.

Pour rappel, nous avons choisi de ne pas utiliser de mutateurs pour la voiture et son conducteur car il faut gérer de manière plus fine la relation entre les deux classes. En effet, il ne faudrait pas se retrouver avec, par exemple, une personne

p qui conduit une voiture v , mais cette voiture v n'aurait aucun conducteur affecté ou bien un autre conducteur p' ! Pour gérer la relation bidirectionnelle, on définit une classe "maître" qui va gérer en priorité cette relation. Ici nous choisissons par exemple la classe **Personne**. Dans cette classe, nous définissons des règles métiers *affectation* et *restitution* d'une voiture pour bien gérer l'association bidirectionnelle Personne-Voiture. Ainsi, dans le cas de l'association bidirectionnelle, pour pouvoir affecter une voiture à une personne :

- la voiture doit exister ;
- la voiture doit être disponible (pas encore de conducteur affecté à la voiture) ;
- la personne doit être un piéton (pas encore de voiture affectée).

Si ces trois conditions sont vérifiées alors :

- on affecte la voiture à la personne ;
- et on définit la personne comme étant conducteur de la voiture.

Ainsi, du côté de la voiture, lorsqu'on lui affecte un conducteur, ce conducteur doit déjà être déclaré comme conducteur de la voiture. Et donc pour affecter un conducteur à une voiture :

- la personne doit exister ;
- la voiture doit être disponible (par encore de conducteur affecté) ;
- le conducteur doit conduire la voiture à laquelle on l'affecte (on peut tester l'égalité des immatriculations).

Question 19. Proposez un diagramme de séquence pour l'affectation d'une voiture à une personne. Vérifiez que la relation entre les classes **Personne** et **Voiture** est toujours préservée.

Question 20. Dans la classe **Personne** modifiez la méthode **public boolean affecter(Voiture v)** afin qu'elle gère à présent l'association bidirectionnelle. N'hésitez pas à faire des affichages dans vos méthodes pour voir ce qui se passe. Faites des tests pour vérifier que tout se déroule correctement. En particulier, vérifiez ce qui se passe dans les cas suivants :

- sur une personne, on affecte une voiture qui vaut **null** ;
- sur une personne, on affecte une voiture qui a déjà un conducteur ;
- sur une personne qui conduit déjà une voiture, on affecte une voiture ;
- sur une voiture, on met un conducteur qui vaut **null** ;

- sur une voiture, on met un conducteur qui n'a pas de voiture affectée ;
- sur une voiture qui a déjà un conducteur affecté, on met un conducteur ;
- sur une voiture, on met un conducteur qui conduit déjà une autre voiture.

Maintenant que nous savons affecter des voitures à des personnes avec l'association bidirectionnelle, il nous faut pouvoir restituer une voiture, tout en conservant bien la relation entre les deux classes, i.e. si la personne n'a plus de voiture, alors la voiture n'a plus de conducteur. De même que pour l'affectation, nous allons supposer ici que la classe **Personne** est la classe "maître" qui va gérer en priorité la restitution.

Ainsi, pour pouvoir restituer la voiture d'une personne, on s'assure d'abord que la personne n'est pas piéton. Puis on restitue la voiture, et la personne devient donc piéton, et on demande à la voiture de libérer son conducteur. Du côté de la voiture, pour libérer le conducteur, on s'assure que la voiture n'est pas disponible (elle a bien un conducteur), et que le conducteur de la voiture est piéton. Alors dans ce cas on rend la voiture disponible.

Question 21. Proposez un diagramme de séquence pour la restitution d'une voiture par une personne. Vérifiez que la relation entre les classes **Personne** et **Voiture** est toujours préservée.

Question 22. Dans la classe **Personne** modifiez la méthode **public boolean restituer()** afin qu'elle prenne en compte l'association bidirectionnelle. Faites des tests pour vérifier que tout se déroule correctement.